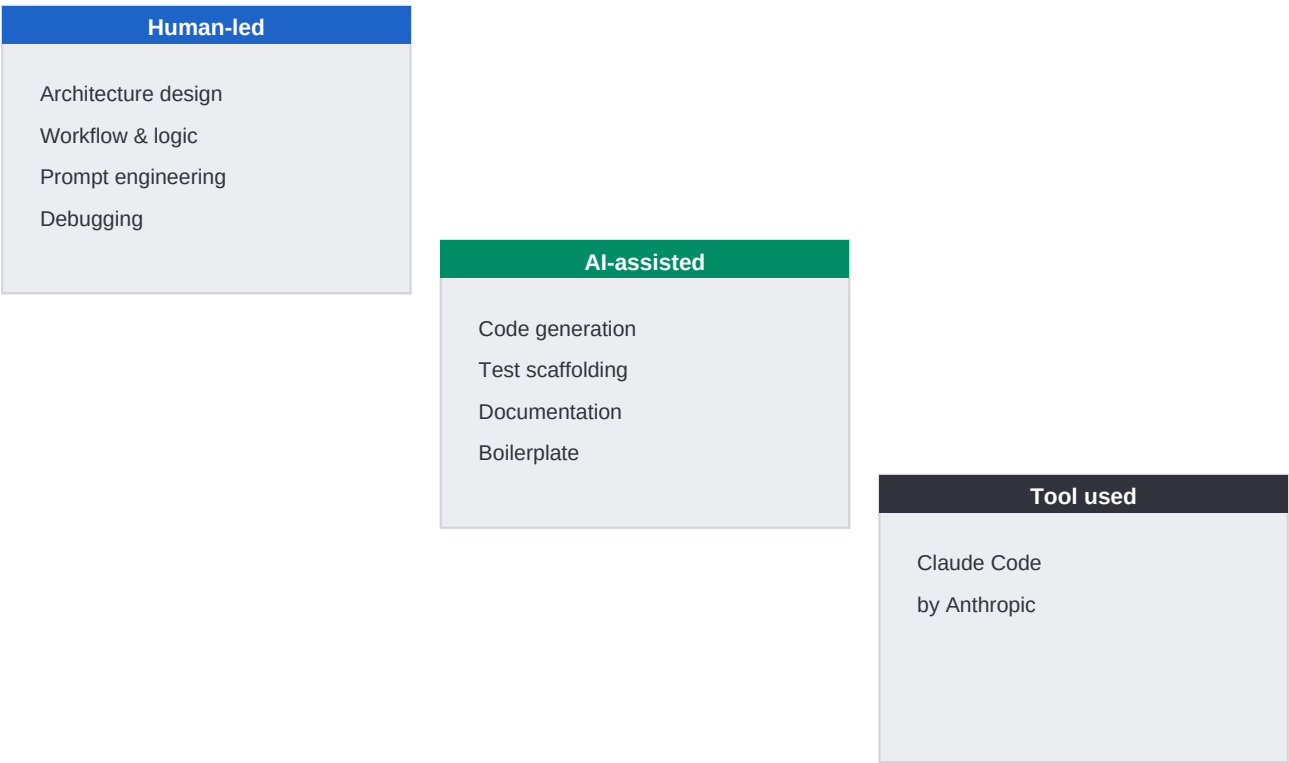


AI Tooling & Human Collaboration

How Claude Code Assisted the Development of the Drone Security System

This document describes how AI-assisted tooling -- specifically Claude Code by Anthropic -- was used throughout the development of the Drone Security System. It explains the division of responsibility between human-directed design work and AI-assisted code generation, and reflects honestly on what each side contributed.



1. Role Division: Human vs. AI

My responsibilities (human-led)

- Defining the overall product goal and the problem to be solved.
- Designing the system architecture: the four-layer pipeline (Ingestion -> Processing -> Storage -> Delivery), the choice of LangGraph for agent orchestration, and the dual-store memory strategy (SQLite + ChromaDB).
- Designing the LangGraph workflow: identifying the six nodes (perceive, contextualize, rule_check, llm_judge, alert, log), deciding the conditional routing logic (needs_llm flag), and specifying what each node should read and write to AgentState.
- Writing and iterating on prompts: the VLM captioning prompt, the LLM judge prompt (VERDICT / ALERT / REASON format), the RAG answer prompt, the query intent parser prompt, and the follow-up suggestion prompt.
- Debugging logic and code: diagnosing the SQLite :memory: connection isolation bug, the ChatBedrock instantiation timing issue in tests, and the two-invoke pattern needed to trigger loitering rules in integration tests.
- Validating outputs at every stage and deciding when generated code was correct, when it needed adjustment, and when the approach needed rethinking.
- Writing all YAML rule definitions -- the condition schemas, severity levels, and needs_llm escalation flags for 20+ security rules.
- Making all product decisions: telemetry simulator design, alert cooldown strategy, frame storage resolution separation, async VLM threading model.

Claude Code's contributions (AI-assisted)

- Generating the implementation code for modules once the design was specified: SQLiteStore, ChromaStore, HybridRetriever, VideoIngestor, FramePreprocessor, TelemetrySimulator, StreamProcessor, RuleEngine, and all LangGraph node factories.
- Writing the full Streamlit dashboard layout -- tab structure, CSS styling, the Live Feed result loop, Timeline event expanders, and the Ask tab RAG chat interface.
- Writing the 124-test suite across three files (test_sqlite_store.py, test_hybrid_retriever.py, test_agent_pipeline.py) once test scenarios were described.
- Generating documentation files: README.md, docs/architecture.md, docs/design-decisions.md, docs/TESTING.md, docs/feature-spec.md, and the system flow diagrams.
- Implementing boilerplate and repetitive structures (TypedDict definitions, factory function patterns, context manager wrappers) that follow from the design.

2. Where Human Judgment Was Most Critical

Workflow design

The decision to use LangGraph rather than a simple function chain was mine. I identified that the pipeline needed conditional routing -- clean frames should skip alerting entirely, ambiguous rule hits should escalate to the LLM, and every frame regardless of outcome should be logged. A linear chain cannot express this cleanly. I specified the node boundary contract (AgentState TypedDict) and the routing function signature before any code was written.

Prompt engineering

The LLM judge prompt required several iterations. The initial version returned free-form text; I restructured it to enforce a rigid three-field format (VERDICT: / ALERT: / REASON:) so the parsing logic could be deterministic. The query intent parser prompt evolved significantly -- I added explicit mapping rules for class name synonyms (people/person/pedestrian -> 'person'), the instruction to return null rather than an empty string, and the telemetry filter fields (battery_below, altitude_above, flight_mode_filter) as requirements grew. Getting the LLM to return clean JSON without markdown fences required explicit negative instructions in the prompt.

Debugging

Three non-trivial bugs required diagnosis before Claude Code could fix them. First, all in-memory SQLite tests were failing with 'no such table: frames' -- I identified that `sqlite3.connect(':memory:')` creates a fresh empty database on every call, so the schema applied in `__init__` was invisible to subsequent queries. The fix (caching `self._mem_conn` for in-memory databases) was my diagnosis. Second, the LLM judge tests were failing because ChatBedrock was instantiated at graph-build time, not at invoke time, so patching after `build_agent()` was called had no effect. I identified the instantiation timing issue and directed the fix. Third, loitering tests never triggered because the `contextualize` node's internal `_track_first_seen` closure resets on every new graph invocation -- I designed the two-invoke test pattern (seed the track, then invoke 65 seconds later) to correctly exercise the accumulation logic.

Architecture decisions

Every major architectural choice was mine: the stationary-drone assumption and its documented implications for future GPS geofencing, the hybrid SQL + CLIP retrieval routing strategy, the async VLM thread pool design, the `drop_on_full=True` vs. `False` distinction between RTSP and file sources, and the alert deduplication cooldown mechanism. Claude Code implemented these decisions once they were specified, but did not originate them.

3. Specific Claude Code Interactions

Prompt / task given to Claude Code:

Design the LangGraph pipeline with six nodes and conditional routing based on a `needs_llm` flag.

What I specified: Node names, `AgentState` fields each node reads/writes, routing rules (`needs_llm=True -> llm_judge, False -> alert, always -> log`), and the factory function pattern (`make_perceive_node` returns `perceive`).

What AI generated: Full implementation of `nodes.py` including all six factory functions, the `_summarise_detections` and `_extract_field` helpers, and the async VLM closure pattern.

What I adjusted: Adjusted the `perceive` node to extract telemetry from `packet.metadata` and return it in state. Added the `telemetry_summary` block to the `contextualize` node's context string. Reviewed and corrected the cooldown logic in the `alert` node.

Prompt / task given to Claude Code:

Implement `TelemetrySimulator` with flight phase timeline, realistic battery/altitude/GPS/signal models, and named scenarios.

What I specified: The field names, the phase sequence (`TAKEOFF -> PATROL -> HOVER -> RETURN -> LANDING`), the discharge model (% per minute), circular GPS orbit, `get_telemetry(ts)` API, and the four named scenarios with their parameter sets.

What AI generated: Full implementation of `simulator.py` including all phase interpolation methods, ease-in/ease-out curves, GPS orbit calculation, battery temperature model, and signal degradation with 2% interference probability.

What I adjusted: Added the `telemetry_summary` string key to the output dict so it could be injected directly into the chatbot context. Verified the phase timeline boundary conditions.

Prompt / task given to Claude Code:

Write 124 tests covering SQLiteStore, HybridRetriever, and the full LangGraph pipeline.

What I specified: The test scenarios (loitering two-invoke pattern, LLM judge false positive path, SQL fallback, cooldown deduplication), the stub model classes, and the diagnosis of the three bugs described above.

What AI generated: All 124 test functions, fixture setup, the StubDetector / StubCaptioner / StubEmbedder classes, and the `_make_preprocessed` helper.

What I adjusted: Directed the fix for the `:memory:` connection bug. Moved `build_agent()` inside the ChatBedrock patch context. Rewrote the loitering test to use the two-invoke pattern after diagnosing why it never triggered.

Prompt / task given to Claude Code:

Build the Streamlit dashboard with Live Feed, Timeline, and Ask tabs.

What I specified: The three-tab layout, the Live tab's dual-column structure (video + alerts), the telemetry strip widget design, the Ask tab's intent-parse -> retrieve -> answer flow, and all prompt text for the LLM calls.

What AI generated: The full `streamlit_app.py` including CSS styling, the StreamProcessor integration loop, the `_parse_query_intent` / `_build_smart_context` / `_bedrock_rag_answer` functions, and the source frames expander.

What I adjusted: Added the telemetry scenario selector to the sidebar. Designed the telemetry metrics strip HTML. Extended `_parse_query_intent` with telemetry filter fields after deciding the chatbot should handle battery/altitude queries.

4. Honest Assessment

Claude Code generated the majority of the implementation code in this project. That is an honest statement and it reflects how modern AI-assisted development works in practice.

What I brought to this project was the engineering judgment that made the code worth generating: the system design, the workflow logic, the identification of edge cases and failure modes, the prompt engineering that made the LLM components reliable, and the debugging that fixed the non-obvious issues that no amount of code generation would have resolved on its own.

AI code generation is most useful -- and most dangerous -- when the human directing it understands what correct looks like. In this project, the design decisions (LangGraph routing, hybrid retrieval, async VLM, stationary-drone assumption) required domain reasoning that the AI did not originate. The implementation of those decisions, once fully specified, was generated efficiently by Claude Code.

The skill being demonstrated here is not the ability to write every line by hand -- it is the ability to design a non-trivial system, decompose it into well-specified components, direct an AI tool to implement them correctly, and identify and fix the cases where the generated output is wrong.